

INSTITUTE OF ACTUARIES OF INDIA

CS2B – Risk Modelling and Survival Analysis (Paper B)

November 2025 Examination

Indicative Solution

Introduction:

The indicative solution has been written by the Examiners with the aim of helping candidates. The solutions given are only indicative. It is realized that there could be other points as valid answers and examiners have given credit for any alternative approach or interpretation which they consider to be reasonable.

Solution 1:**i)***#Importing libraries*

library(dplyr)

Reading the data into a data frame

claims=read.csv("Q1_Data.csv",header=T,sep=",")

Formatting the date column

claims\$Claim_Date <- as.Date(claims\$Claim_Date, format("%d-%m-%Y"))

Printing the first 6 rows

head(claims)

Output:

	Policy_No	Claim_Date	Claim_Cost	Line_Of_Business
1	6498550	2024-03-29	17101	Property
2	4481695	2019-03-21	4189	Motor
3	8500459	2024-09-12	4825	Property
4	8332774	2021-12-18	13614	Property
5	9973400	2019-07-30	4989	Property
6	1862312	2018-11-19	424	Health

(2)**ii)***# Checking for Missing values*

colSums(is.na(claims))

Output:

Policy_No	Claim_Date	Claim_Cost	Line_Of_Business
0	0	0	0

As there are no missing values, no adjustments are needed.

Checking for duplicates

sum(duplicated(claims))

Output:

[1] 6

Removing duplicates

claims <- unique(claims)

Checking for incorrect dates

```
claims %>% filter(Claim_Date>as.Date("2025-07-31") |
Claim_Date<as.Date("2014-11-01"))
```

Output:

	Policy_No	Claim_Date	Claim_Cost	Line_Of_Business
1	5056474	2006-12-11	3715	Motor
2	9894368	2029-02-05	3949	Motor

```
claims <- claims %>% filter(Claim_Date<as.Date("2025-07-31") &
Claim_Date>as.Date("2014-11-01"))
```

(5)

iii)

```
# Sort the claims by claim date
```

```
claims <- arrange(claims,Claim_Date)
```

```
# Check if the claims are sorted by claim date
```

```
head(claims)
```

Output:

	Policy_No	Claim_Date	Claim_Cost	Line_Of_Business
1	4432195	2014-11-06	4297	Motor
2	5939493	2014-11-27	3131	Motor
3	5795563	2015-01-15	3775	Motor
4	8926179	2015-01-20	13215	Property
5	5443494	2015-02-23	4142	Motor
6	6840703	2015-03-12	2146	Motor

(1)

iv)

```
# Running the function to calculate waiting periods
```

```
add_waiting_period <- function (data) {
  temp <- cbind(data,c(NA,diff(as.numeric(data[,2]))))
  colnames(temp)[5] <- "Waiting_Period"
  temp
}
```

Explanation of the function:

- The function takes the argument data of data frame format
- The function diff calculates difference between consecutive vector elements
- The output generated by using the diff function on 2nd column,
- is added as a new column to the data frame
- This column (5th column) is then renamed as "Waiting_Period"
- The resulting data frame with the new column is printed in the end.

(2)

v)

```
# Using the function
```

```
claims <- add_waiting_period(claims)
head(claims)
```

Output:

	Policy_No	Claim_Date	Claim_Cost	Line_Of_Business	Waiting_Period
1	4432195	2014-11-06	4297	Motor	NA
2	5939493	2014-11-27	3131	Motor	0.05749487
3	5795563	2015-01-15	3775	Motor	0.13415469
4	8926179	2015-01-20	13215	Property	0.01368925
5	5443494	2015-02-23	4142	Motor	0.09308693

```
6 6840703 2015-03-12 2146 Motor 0.04654346
```

```
# Calculating the poisson parameter for all claims
```

```
mean_claim_time=mean(claims[,5][2:length(claims[,5])])
Lambda_all=1/mean_claim_time
Lambda_all
```

Output:

```
[1] 65.46404
```

(2)

vi)

```
# Creating datasets for each lob
```

```
motor <- claims[claims$Line_Of_Business=="Motor",]
property <- claims[claims$Line_Of_Business=="Property",]
health <- claims[claims$Line_Of_Business=="Health",]
```

```
# Printing the datasets
```

```
head(property)
```

Output:

	Policy_No	Claim_Date	Claim_Cost	Line_Of_Business	Waiting_Period
4	8926179	2015-01-20	13215	Property	0.013689254
7	9409314	2015-03-25	8768	Property	0.035592060
8	4070818	2015-04-01	11467	Property	0.019164956
9	7096805	2015-04-02	6266	Property	0.002737851
18	8405049	2015-06-23	17339	Property	0.019164956
23	6523198	2015-09-10	10027	Property	0.035592060

```
# Printing number of claims in each line of business
```

```
nrow(property)
```

Output:

```
[1] 219
```

OR

```
table(claims$Line_Of_Business)
```

Output:

Health	Motor	Property
135	346	219

(3)

vii)

```
# Calculating waiting periods for property claims
```

```
property <- arrange(property, Claim_Date)
property <- add_waiting_period(property)
head(property)
```

Output:

	Policy_No	Claim_Date	Claim_Cost	Line_Of_Business	Waiting_Period
1	8926179	2015-01-20	13215	Property	0.013689254
2	9409314	2015-03-25	8768	Property	0.035592060

```

3 4070818 2015-04-01 11467 Property 0.019164956
4 7096805 2015-04-02 6266 Property 0.002737851
5 8405049 2015-06-23 17339 Property 0.019164956
6 6523198 2015-09-10 10027 Property 0.035592060
c(NA, diff(as.numeric(data[, 2])))/365.25
1 NA
2 0.175222450
3 0.019164956
4 0.002737851
5 0.224503765
6 0.216290212

```

Calculating the poisson parameter for Property claims

```

mean_claim_time=mean(property[,6][2:length(property[,6])])
Lambda_prop=1/mean_claim_time
Lambda_prop

```

Output:

```
[1] 21.24453
```

(2)

viii)

Calculation the poisson parameter for Property claims by thinning

```

proportion_prop=length(property[,5])/length(claims[,5])
Lambda_prop_thinning=Lambda_all*proportion_prop
Lambda_prop_thinning

```

Output:

```
[1] 20.48089
```

Comment:

The Poisson parameters derived from both the methods are quite close.

(2)

ix)

Sample mean

```

mean_claim_cost <- mean(property$Claim_Cost)
mean_claim_cost

```

Output:

```
[1] 10947.77
```

Sample standard deviation

```

sd_claim_cost <- sqrt(var(property$Claim_Cost))
sd_claim_cost

```

Output:

```
[1] 5699.061
```

(2)

x)

Estimating alpha and beta for Gamma distribution using method of moments

```
beta = mean_claim_cost/var(property$Claim_Cost)
beta
```

Output:

```
[1] 0.000337069
```

```
alpha = beta*mean_claim_cost
alpha
```

Output:

```
[1] 3.690153
```

(2)

xi)

Calculating expected annual claim amounts for property

```
expected_annual_claims_prop=Lambda_prop*mean_claim_cost
expected_annual_claims_prop
```

Output:

```
[1] 232580.2
```

Calculating standard deviation

```
sd_annual_prop=sqrt(Lambda_prop*((sd_claim_cost)^2+(mean_claim_cost)^2))
sd_annual_prop
```

Output:

```
[1] 56887.97
```

(4)

xii)

Mean Residual Life for Gamma distribution

```
x=seq(500,50000, by = 500)
```

```
S <- function(x) 1 - pgamma(x, shape = alpha, rate = beta)
```

```
MRL_X <- rep(0,100)
```

```
for (i in 1:100) {
```

```
  # Numerator:  $\int$  from t to  $\infty$  S(x) dx
```

```
  num <- integrate(S, lower = x[i], upper = Inf)$value
```

```
  # Denominator: S(t)
```

```
  denom <- S(x[i])
```

```
  # Mean residual life
```

```
  MRL_X[i] <- num / denom
```

```
}
```

```
MRL_X
```

Output:

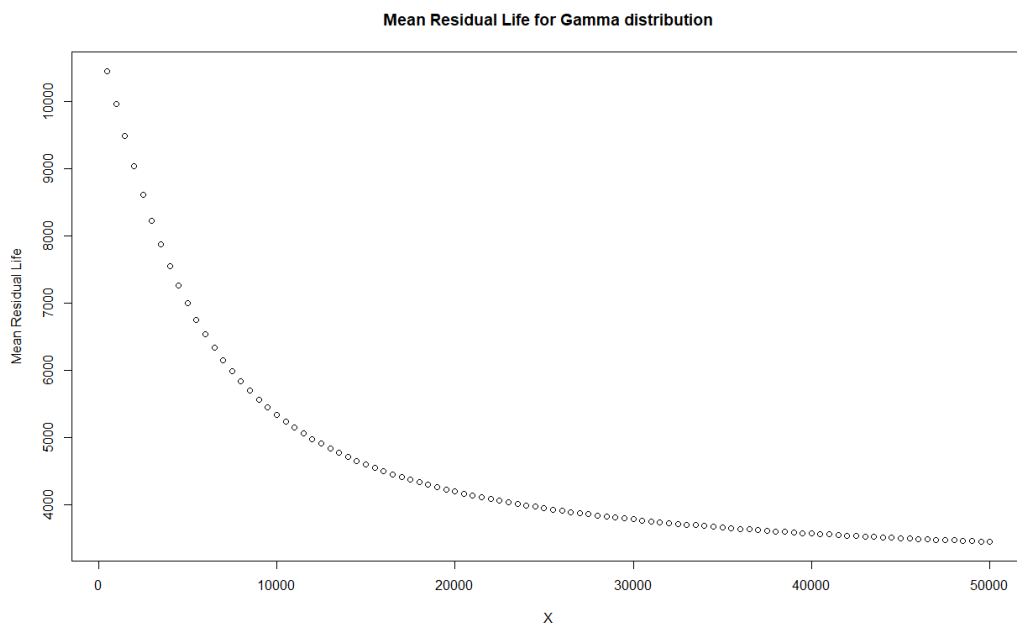
```
[1] 10448.619 9957.063 9482.927 9034.126 8615.548 8229.358 7875.696
7553.374 7260.427 6994.514 6753.176 6533.997 6334.698 6153.182
```

```
[15] 5987.546 5836.089 5697.297 5569.827 5452.494 5344.253 5244.181
5151.464 5065.383 4985.302 4910.658 4840.951 4775.737 4714.621
[29] 4657.217 4603.305 4552.507 4504.599 4459.354 4416.566 4376.047
4337.629 4301.159 4266.498 4233.520 4202.108 4172.159 4143.575
[43] 4116.267 4090.156 4065.165 4041.227 4018.278 3996.260 3975.119
3954.804 3935.269 3916.472 3898.371 3880.931 3864.116 3847.913
[57] 3832.234 3817.109 3802.492 3788.359 3774.686 3761.452 3748.635
3736.218 3724.182 3712.509 3701.185 3690.193 3679.520 3669.152
[71] 3659.077 3649.282 3639.756 3630.489 3621.469 3612.688 3604.136
3595.804 3587.684 3579.769 3572.050 3564.521 3557.174 3550.004
[85] 3543.004 3536.167 3529.490 3522.965 3516.588 3510.354 3504.258
3498.297 3492.464 3486.757 3481.172 3475.703 3470.349 3465.105
[99] 3459.969 3454.936
```

Plot of Mean Residual Life:

```
plot(x=x,y =MRL_X, ,
     xlab="X", ylab="Mean Residual Life",
     main="Mean Residual Life for Gamma distribution")
```

Plot:



Comments:

```
# The Mean residual life decreases with increase in X
# It converges to 3454.9
# It indicates that the Gamma distribution has a lighter tail
```

(6)
[33 Marks]

Solution 2:

i)
Reading the data into a data frame

```
Movie_data=read.csv("Q2_Data.csv",header=TRUE,sep=",")
```

Printing the first 5 rows

```
head(Movie_data,5)
```

Output:

	Year	Movies_Made	Hits
1	2000	677	11
2	2001	1621	15
3	2002	1935	21
4	2003	1999	15
5	2004	1783	13

```
# Printing the last 5 rows
```

```
tail(Movie_data,5)
```

Output:

	Year	Movies_Made	Hits
21	2020	5865	37
22	2021	6254	49
23	2022	6517	30
24	2023	6600	28
25	2024	6528	18

(3)

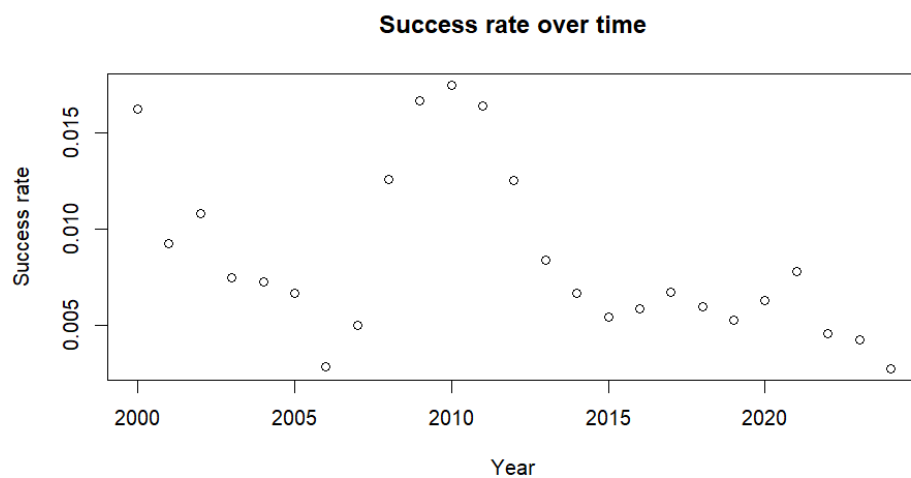
ii)

```
# Calculating the annual claim rates
```

```
Movie_data$Success_Rate <- Movie_data$Hits/Movie_data$Movies_Made
```

```
# Scatterplot
```

```
plot(Movie_data$Success_Rate ~ Movie_data$Year,
     xlab="Year", ylab="Success rate",
     main="Success rate over time")
```

Plot:

(3)

iii)**Comments:**

```
# The success rate shows a crest with peak around 2010.
```



```
# Apart from that, the overall patterns is almost flat.
# There is a decreasing trend in the later years after 2021.
```

(1)

iv)

```
# Fitting Model 1:
```

```
Model_1 <- lm(Success_Rate ~ Year, data=Movie_data)
Model_1
```

Output:

Call:

```
lm(formula = Success_Rate ~ Year, data = Movie_data)
```

Coefficients:

```
(Intercept)      Year
  0.5611667   -0.0002747
```

```
# Predicting the success rate for 2025 and 2026 using Model 1:
```

```
predict(Model_1, data.frame(Year = c(2025,2026)))
```

Output:

```
      1      2
0.004894058 0.004619355
```

(2)

v)

```
# Fitting Model 2:
```

```
Movie_data$Year2 <- Movie_data$Year^2
```

```
Model_2 <- lm(Success_Rate ~ Year+Year2, data=Movie_data)
Model_2
```

Output:

Call:

```
lm(formula = Success_Rate ~ Year + Year2, data = Movie_data)
```

Coefficients:

```
(Intercept)      Year      Year2
-8.671e+01   8.648e-02  -2.156e-05
```

```
# Predicting the success rate for 2025 and 2026 using Model 2:
```

```
predict(Model_2, data.frame(Year = c(2025,2026),
                             Year2 = c(2025^2,2026^2)))
```

Output:

```
      1      2
0.002371675 0.001514884
```

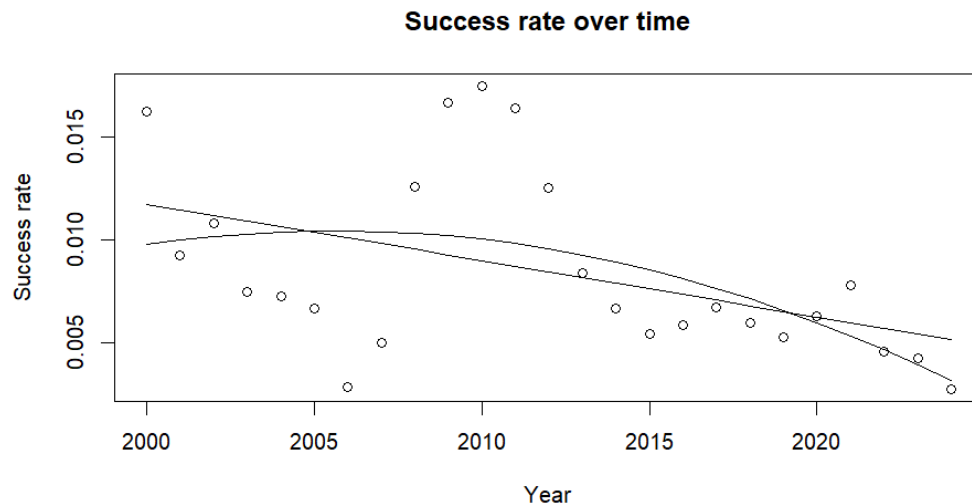
(4)

vi)

```
# Plotting the curves on the scatterplot:
```

```
plot(Movie_data$Success_Rate ~ Movie_data$Year,
     xlab="Year", ylab="Success rate",
     main="Success rate over time")
```

```
lines(Movie_data$Year, Model_1$fitted.values)
lines(Movie_data$Year, Model_2$fitted.values)
```



(2)

vii)

Comments:

```
# Both the model does not fit the data very well
# they underestimate the increase in 2010
# Quadratic polynomial captures the shape a bit better than linear model
# Even the quadratic polynomial doesn't capture the shape completely
# Both the models are not suitable for the whole time period
# Both the models could give negative claim rates in the future.
```

(2)

viii)

Splitting the data into three segments:

```
Movie_data_segment1 <- Movie_data[Movie_data$Year>=2000 &
                                   Movie_data$Year<=2008, ]
Movie_data_segment2 <- Movie_data[Movie_data$Year>=2009 &
                                   Movie_data$Year<=2016, ]
Movie_data_segment3 <- Movie_data[Movie_data$Year>=2017 &
                                   Movie_data$Year<=2024, ]
```

(3)

ix)

Fitting Model S11:

```
Model_S11 <- lm(Success_Rate ~ Year, data=Movie_data_segment1)
Model_S11
```

Output:

Call:

```
lm(formula = Success_Rate ~ Year, data = Movie_data_segment1)
```

```
Coefficients:
(Intercept)      Year
  1.4829515  -0.0007357
```

Fitting Model S21:

```
Model_S21 <- lm(Success_Rate ~ Year, data=Movie_data_segment2)
Model_S21
```

Output:

```
Call:
lm(formula = Success_Rate ~ Year, data = Movie_data_segment2)
```

```
Coefficients:
(Intercept)      Year
  4.058140  -0.002011
```

Fitting Model S31:

```
Model_S31 <- lm(Success_Rate ~ Year, data=Movie_data_segment3)
Model_S31
```

Output:

```
Call:
lm(formula = Success_Rate ~ Year, data = Movie_data_segment3)
```

```
Coefficients:
(Intercept)      Year
  0.8951267  -0.0004403
```

Fitting Model S12:

```
Model_S12 <- lm(Success_Rate ~ Year+Year2, data=Movie_data_segment1)
Model_S12
```

Output:

```
Call:
lm(formula = Success_Rate ~ Year + Year2, data = Movie_data_segment1)
```

```
Coefficients:
(Intercept)      Year      Year2
  1.784e+03  -1.779e+00   4.437e-04
```

Fitting Model S22:

```
Model_S22 <- lm(Success_Rate ~ Year+Year2, data=Movie_data_segment2)
Model_S22
```

Output:

```
Call:
lm(formula = Success_Rate ~ Year + Year2, data = Movie_data_segment2)
```

```
Coefficients:
(Intercept)      Year      Year2
  1.687e+02  -1.657e-01   4.066e-05
```

Fitting Model S32:

```
Model_S32 <- lm(Success_Rate ~ Year+Year2, data=Movie_data_segment3)
Model_S32
```

Output:

Call:

```
lm(formula = Success_Rate ~ Year + Year2, data = Movie_data_segment3)
```

Coefficients:

```
(Intercept)      Year      Year2
-5.779e+02  5.725e-01 -1.418e-04
```

(6)

x)

Determining BIC for each of the models:

```
BIC(Model_S11)
```

Output:

```
[1] -70.57169
```

```
BIC(Model_S21)
```

Output:

```
[1] -75.10967
```

```
BIC(Model_S31)
```

Output:

```
[1] -80.09482
```

```
BIC(Model_S12)
```

Output:

```
[1] -76.82381
```

```
BIC(Model_S22)
```

Output:

```
[1] -73.15484
```

```
BIC(Model_S32)
```

Output:

```
[1] -81.468
```

Determining the best models in each segment based on BIC:

- Model_S12 (quadratic) is the best model for segment 1 as it has lower BIC,
- Model_S21 (linear) is the best model for segment 2 as it has lower BIC,
- Model_S32 (quadratic) is the best model for segment 3 as it has lower BIC.

(5)

xi)

Predicting the success rate for 2025 and 2026 using Model_S32:

```
predict(Model_S32, data.frame(Year = c(2025,2026),
                                Year2 = c(2025^2,2026^2)))
```

Output:

```
      1      2
0.0013592938 -0.0004988777
```

Comments:

The prediction for 2026 is negative which is not possible.

This clearly indicates the limitation of using polynomial regression of this case

The data used in the segmented approach might be insufficient to build a reasonable model

Using alternative approaches like using a logarithmic transformation of the success rate or using generalised linear models instead of quadratic polynomial regression could be useful.

(3)

[34 Marks]**Solution 3:**

i)

a)

Printing both the time series

mdeaths or print(mdeaths)

Output:

```
  Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
1974 2134 1863 1877 1877 1492 1249 1280 1131 1209 1492 1621 1846
1975 2103 2137 2153 1833 1403 1288 1186 1133 1053 1347 1545 2066
1976 2020 2750 2283 1479 1189 1160 1113 970 999 1208 1467 2059
1977 2240 1634 1722 1801 1246 1162 1087 1013 959 1179 1229 1655
1978 2019 2284 1942 1423 1340 1187 1098 1004 970 1140 1110 1812
1979 2263 1820 1846 1531 1215 1075 1056 975 940 1081 1294 1341
```

fdeaths or print(fdeaths)

Output:

```
  Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
1974 901 689 827 677 522 406 441 393 387 582 578 666
1975 830 752 785 664 467 438 421 412 343 440 531 771
1976 767 1141 896 532 447 420 376 330 357 445 546 764
1977 862 660 663 643 502 392 411 348 387 385 411 638
1978 796 853 737 546 530 446 431 362 387 430 425 679
1979 821 785 727 612 478 429 405 379 393 411 487 574
```

i) (b)

Printing the time of the first and last observations for both the time series

```
start(mdeaths)
```

Output:

```
[1] 1974 1
```

```
end(mdeaths)
```

Output:

```
[1] 1979 12
```

```
start(fdeaths)
```

Output:

```
[1] 1974 1
```

```
end(fdeaths)
```

Output:

```
[1] 1979 12
```

(3)

ii)

Plotting the ACF and PACF for both the time series

```
par(mfrow=c(1,2))
```

```
acf(mdeaths,lag.max=60,main="Males - ACF",  
    ylab="Sample ACF",col="red")
```

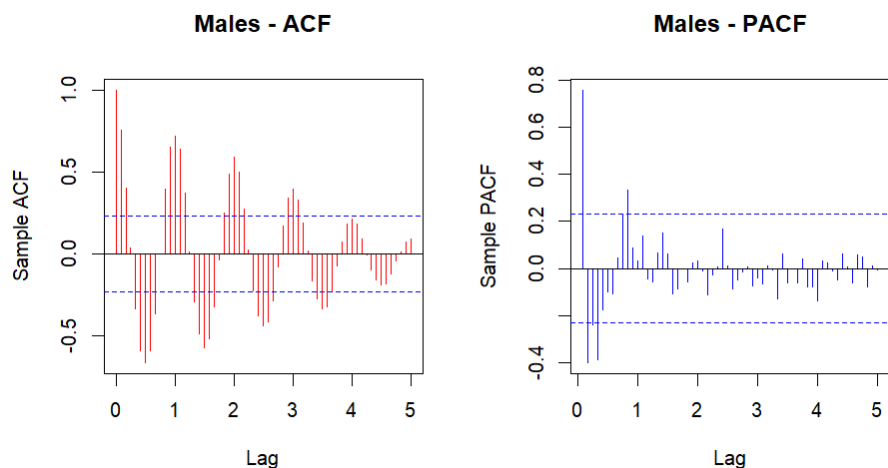
```
pacf(mdeaths,lag.max=60,main="Males - PACF",  
     ylab="Sample PACF",col="blue")
```

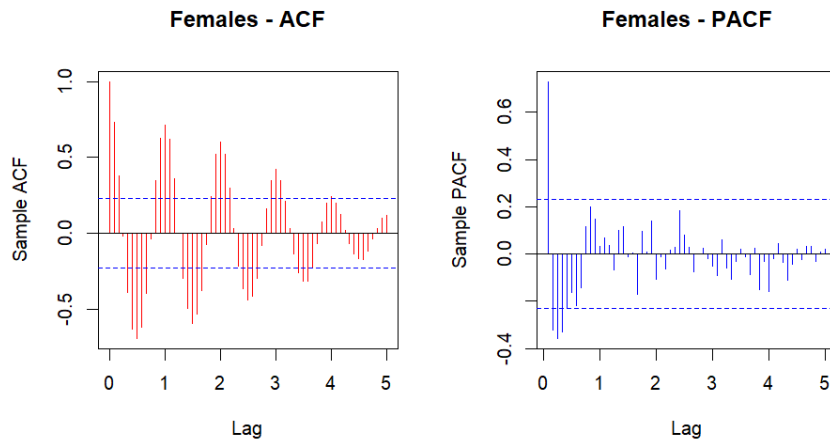
```
acf(fdeaths,lag.max=60,main="Females - ACF",  
    ylab="Sample ACF",col="red")
```

```
pacf(fdeaths,lag.max=60,main="Females - PACF",  
     ylab="Sample PACF",col="blue")
```

```
par(mfrow=c(1,1))
```

Plots:





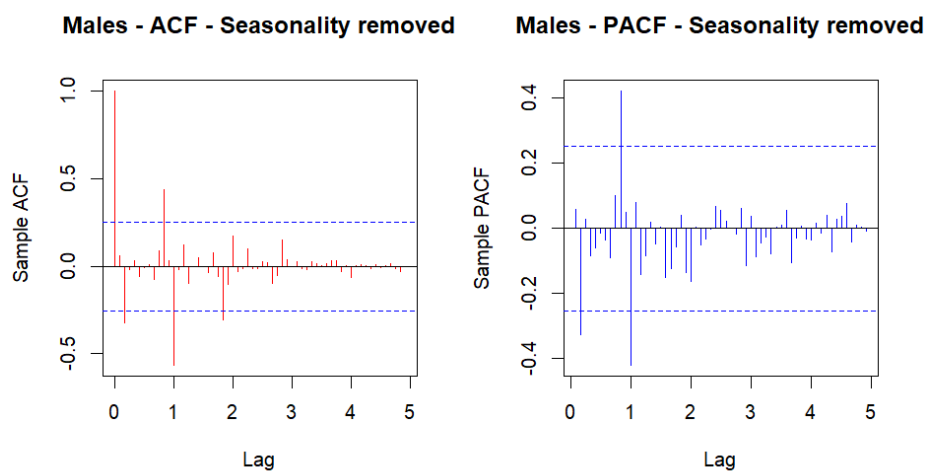
(5)

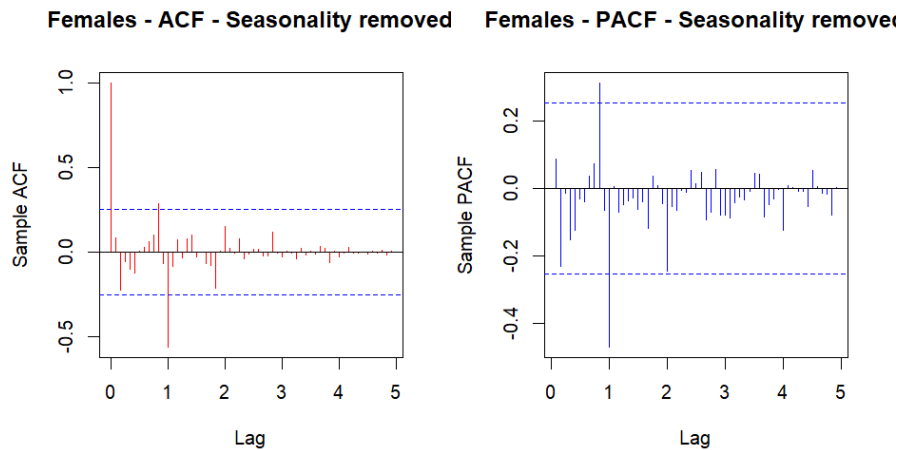
iii)

Plotting the ACF and PACF for both the time series (after removing seasonality)

```
acf(diff(mdeaths, lag=12),lag.max=60,main="Males - ACF - Seasonality
removed",
  ylab="Sample ACF",col="red")
pacf(diff(mdeaths, lag =12),lag.max=60,main="Males - PACF - Seasonality
removed",
  ylab="Sample PACF",col="blue")
acf(diff(fdeaths, lag=12),lag.max=60,main="Females - ACF - Seasonality
removed",
  ylab="Sample ACF",col="red")
pacf(diff(fdeaths, lag=12),lag.max=60,main="Females - PACF - Seasonality
removed",
  ylab="Sample PACF",col="blue")
```

Plots:





(4)

iv)

Comment:

The ACF drops to 0 after lag 1 for both time series after the seasonality
 # is removed by seasonal differencing, while the PACF gradually reduces to 0.
 # This strongly suggests that both the time series are MA (1) time series
 # after seasonal differencing. in terms of ARIMA, it is ARIMA (0,0,1)

(2)

v)

Writing a loop to determine the ARIMA (p, d, q) model with the least AIC.

```
for (p in 0:1) for (d in 0:1) for (q in 0:1) {
  aic<-arima(fdeaths,order=c(p,d,q),seasonal = list(order = c(0,1,1), period =
12))$aic
  row<-c(p,d,q,aic)
  print(row)
}
```

Output:

```
[1] 0.0000 0.0000 0.0000 708.3114
[1] 0.0000 0.0000 1.0000 708.9819
[1] 0.0000 1.0000 0.0000 728.1896
[1] 0.0000 1.0000 1.0000 700.9840
[1] 1.0000 0.0000 0.0000 709.3251
[1] 1.0000 0.0000 1.0000 710.3756
[1] 1.0000 1.0000 0.0000 722.9935
[1] 1.0000 1.0000 1.0000 702.4406
```

The model with least AIC is ARIMA (0,1,1)

(6)

vi)

Fitting ARIMA (0, 1, 1) models to both the datasets

```
males_fit<-arima(mdeaths,order=c(0,1,1),seasonal = list(order = c(0,1,1), period =
12))
males_fit
```

Output:


```

Call:
arima(x = mdeaths, order = c(0, 1, 1), seasonal = list(order = c(0, 1, 1), period = 12))

Coefficients:
      ma1      sma1
    -1.0000 -1.0000
s.e.  0.1444  0.2656

sigma^2 estimated as 28231: log likelihood = -399.46, aic = 804.92

females_fit<-arima(fdeaths,order=c(0,1,1),seasonal = list(order = c(0,1,1), period =
12))
females_fit

```

```

Output:
Call:
arima(x = fdeaths, order = c(0, 1, 1), seasonal = list(order = c(0, 1, 1), period = 12))

Coefficients:
      ma1      sma1
    -1.0000 -1.0000
s.e.  0.0818  0.2628

sigma^2 estimated as 4849: log likelihood = -347.49, aic = 700.98

```

(2)

vii)

Performing the Ljung and Box portmanteau test to check the fit of the models

Null Hypothesis: the residuals are independent

Alternate Hypothesis: the residuals are not independent

```
Box.test(males_fit$residuals,type="Ljung",fitdf=2,lag=14)
```

```

Output:
      Box-Ljung test

data: males_fit$residuals
X-squared = 19.041, df = 12, p-value = 0.08755

```

```
Box.test(females_fit$residuals,type="Ljung",fitdf=2,lag=14)
```

```

Output:
      Box-Ljung test

data: females_fit$residuals
X-squared = 8.7185, df = 12, p-value = 0.7268

```

Outcome:

p-value for mdeaths is $0.08755 > 0.05$ and for fdeaths is $0.7268 > 0.05$.

In both the cases as p-values are greater than 5% indicating the null hypothesis that the residuals are independent cannot be rejected.

Hence the ARIMA (0, 1, 1) models are a good fit for both the datasets.

(5)

viii)

```
round(predict(males_fit,n.ahead=12)$pred)
```

Output:

```
Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
1980 1957 1909 1798 1485 1142 1014 964 865 849 1069 1205 1624
```

```
round(predict(females_fit,n.ahead=12)$pred)
```

Output:

```
Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec
1980 794 778 737 577 456 386 379 335 340 413 461 647
```

(2)

ix)

Performing Phillips-Perron Unit Test to check whether the cointegrated time series is stationary

Null hypothesis: the cointegrated series has a unit root (it can be differenced)

Alternative hypothesis: the cointegrated series cannot be differenced (it is stationary)

```
PP.test(0.90*mdeaths+1.09*fdeaths)
```

Output:

Phillips-Perron Unit Root Test

data: 0.9 * mdeaths + 1.09 * fdeaths

Dickey-Fuller = -3.8343, Truncation lag parameter = 3, p-value = 0.02213

p-value is $0.02213 < 5\%$. So, we have sufficient evidence to reject the null hypothesis and conclude that the cointegrated time series with the cointegrating vector (0.90, 1.09) is stationary.

(4)

[33 Marks]
